

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 28

Выполнил: Лапташкин В.И.

Группа: А-08-23

Вариант: 12

Проверил: Гольцов А.Г.

Инф. № подл.	Подп. и дата
Взам. инф. №	Инф. № дубл.
Подп. и дата	Подп. и дата

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 12 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1528 байт, выработка stroba Y_1 длительностью 298 мс, выработка сигнала Y_2 со стробом 2068 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1. Общие сведения.....	4
2. Функциональное назначение.....	5
2.1. Классы решаемых задач.....	5
2.2. Назначение программы.....	5
2.3. Сведения о функциональных ограничениях на применение.....	5
3. Описание логической структуры.....	6
3.1. Алгоритм программы.....	6
3.2. Структура программы с описанием функций составных частей и связи между ними.....	18
3.3. Связи программы с другими программами.....	19
4. Входные данные.....	20
4.1. Клавиатура.....	20
4.2. Сигнал X2.....	20
5. Выходные данные.....	21
5.1. Характер и организация выходных данных.....	21
5.2. Формат, описание и способ кодирования выходных данных.....	21
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	23
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	24
ПРИЛОЖЕНИЕ А. Листинги программ.....	25

1. ОБЩИЕ СВЕДЕНИЯ

Обозначение и наименование программы — «Программа управления объектом», вариант 12, группа А-08-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Среда разработки — KEIL μ Vision V5; ассемблер — A51, компоновщик — BL51 из пакета PK51 (Keil Software). Пошаговая отладка выполнялась в симуляторе среды.

2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1. Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2. Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — чтение из кольцевого буфера (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — вывод символа по коду на индикатор с гашением через T_3 (прерывание таймера).

2.3. Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 4×3 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 2068 мкс на выводе P1.6;
- выходной сигнал Y_1 — строб низкого уровня длительностью 298 мс на выводе P1.5;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более $T_3 = 2,820$ с.

3. ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1. Алгоритм программы

После сброса МПС выполняет инициализацию (рисунок 3.1): ставит стек на 07h, включает дешифратор адреса линией CS_EN, опускает все столбцы клавиатуры, гасит индикатор, задаёт режимы таймеров и разрешает прерывания. Дальше всю срочную работу делают обработчики: INT0 срабатывает по нажатию клавиши и вызывает опрос матрицы, INT1 по стробу X2 дожидается конца строба, забирает из ячейки обмена отсчёт, который внешнее устройство записало туда правым портом IDT7005, и кладёт его в кольцевой буфер, Timer0 отсчитывает длительности T1 (строб Y1) и T3 (время показа символа), а Timer1 — длительность строба Y2.

Основной цикл (рисунок 3.2) только следит за условием выдачи Y2: если $X1 \neq 0$, в буфере есть отсчёты и предыдущий строб Y2 закончился, он забирает отсчёт из буфера, считает $Y2 = (20 + X1 + X2) \bmod 256$ и выдаёт его со стробом; при $X1 = 0$ в регистр один раз записывается ноль. Три программы для робота — это отдельные куски этой системы: обработчик клавиатуры (рисунок 3.3), чтение из буфера (рисунок 3.7) и вывод символа с гашением по T3 (рисунки 3.9 и 3.10); в них сделана только своя часть инициализации, остальное время МК стоит на команде jmp \$.

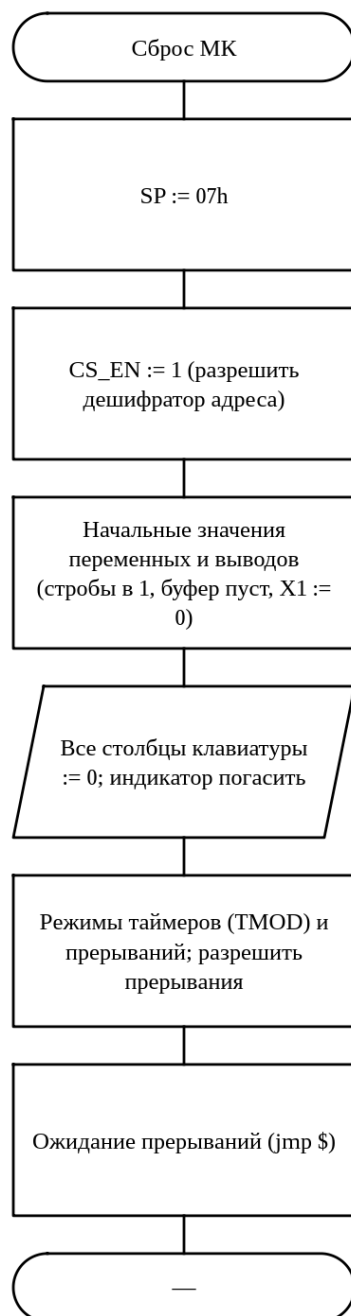


Рис. 3.1. Схема алгоритма: Инициализация МПС и основная программа

Сначала задаётся $SP = 07h$, чтобы стек лежал сразу за банком регистров 0, и ставится $CS_EN = 1$ — без этого дешифратор 74HC138 не выдаст ни одного CS и команды movh никуда не попадут. Затем переменные получают начальные значения: буфер пуст (голова = хвост = 6000h, $F_EMPTY = 1$), $X1 = 0$, стробы Y1 и Y2 в единице. В регистр столбцов пишется 0, чтобы любое нажатие сразу опустило строку и дало спад на INT0, а индикатор гасится кодом FFh. Последними настраиваются таймеры и разрешаются прерывания: если разрешить их раньше, обработчик мог бы застать недонастроенные переменные.

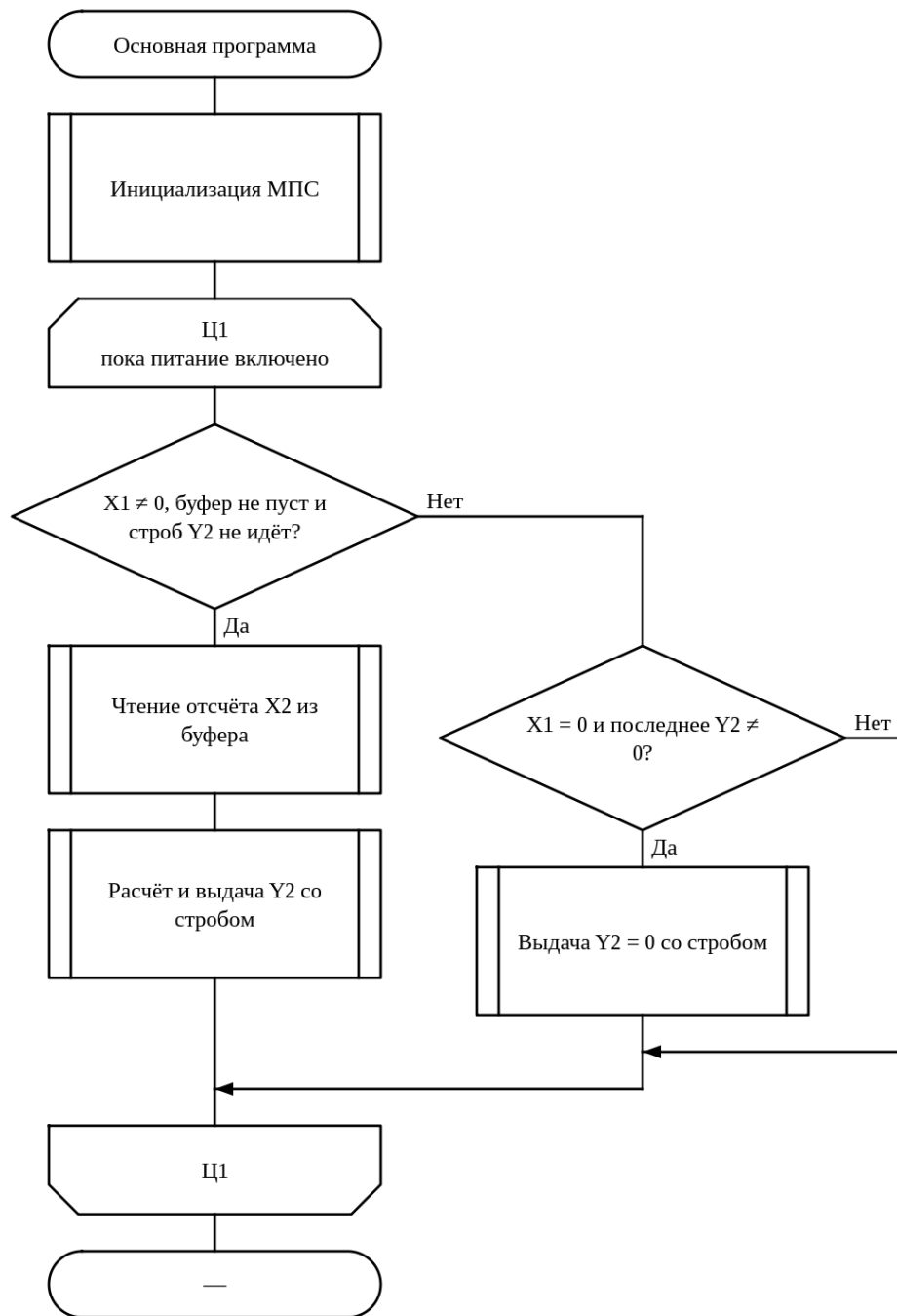


Рис. 3.2. Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие « $X1 \neq 0$, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие « $X1 = 0$ и последнее Y2 $\neq 0$ »: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

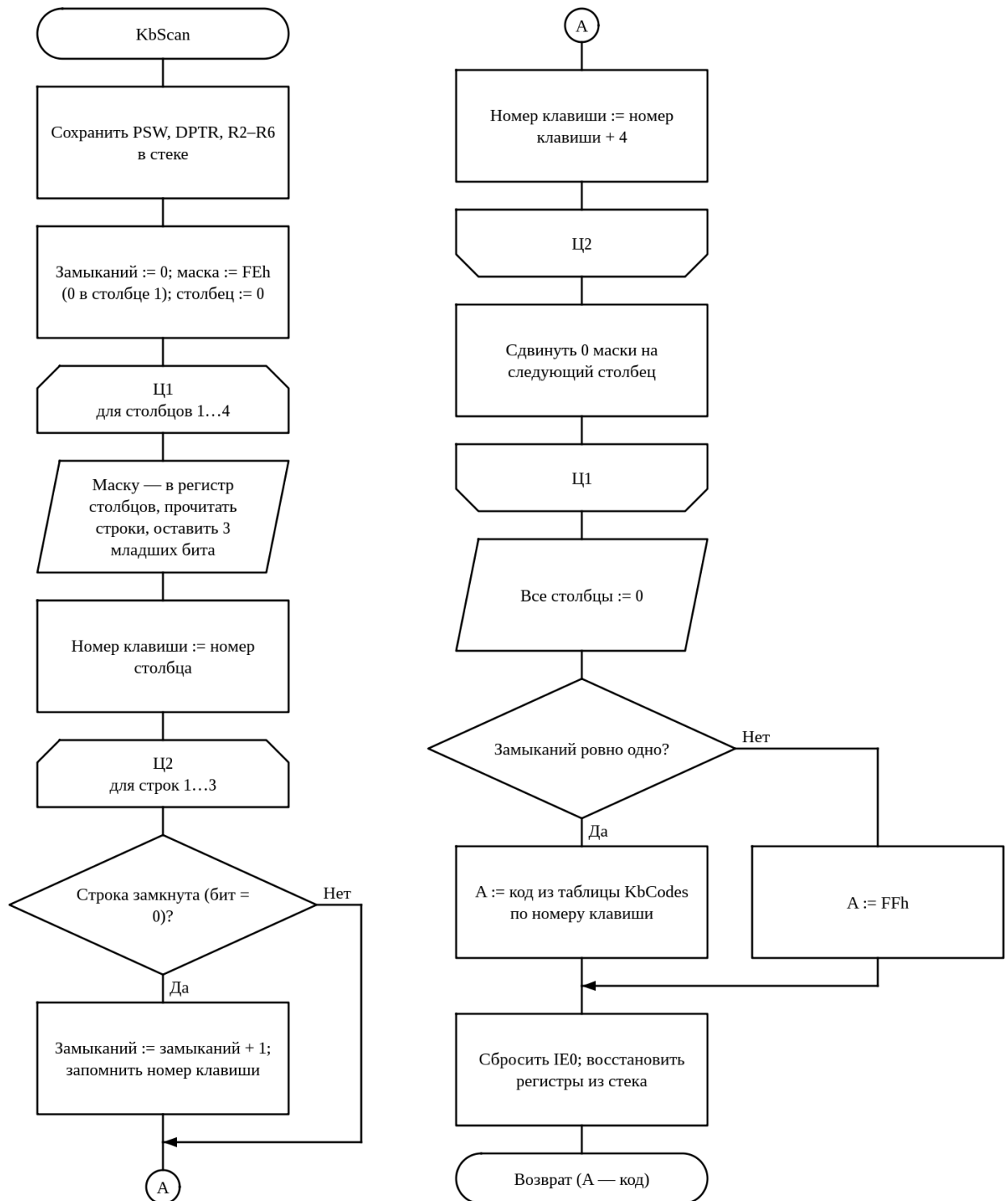


Рис. 3.3. Схема алгоритма: Опрос клавиатуры (обработчик INT0)

Опрос идёт по столбцам: маска с одним нулём записывается в регистр столбцов, и из ячейки клавиатуры читаются три младших бита — состояние строк, старшие биты шины отбрасываются. Внутри столбца строки перебираются сдвигом через перенос, номер клавиши при переходе на строку ниже увеличивается на 4 (число столбцов). Я не останавливаюсь на первом найденном контакте, а считую все замыкания: только если за весь опрос нашлось ровно одно, номер клавиши переводится в код по таблице KbCodes, иначе (помеха или две клавиши сразу) возвращается FFh. В конце все столбцы снова опускаются в 0, а IE0 сбрасывается — пока перебирались столбцы, линия INT0 дёргалась, и без сброса обработчик вызвался бы ещё раз.

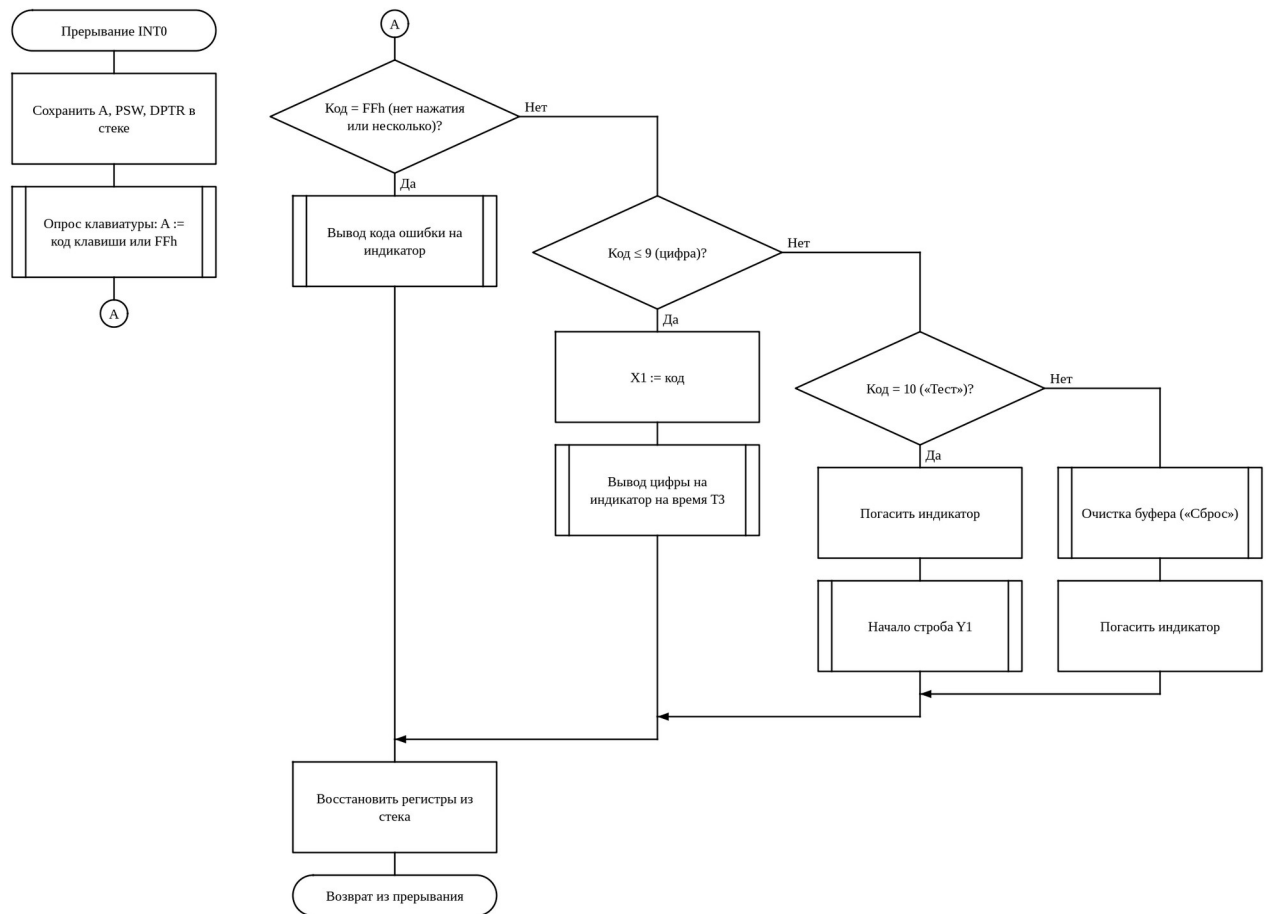


Рис. 3.4. Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.

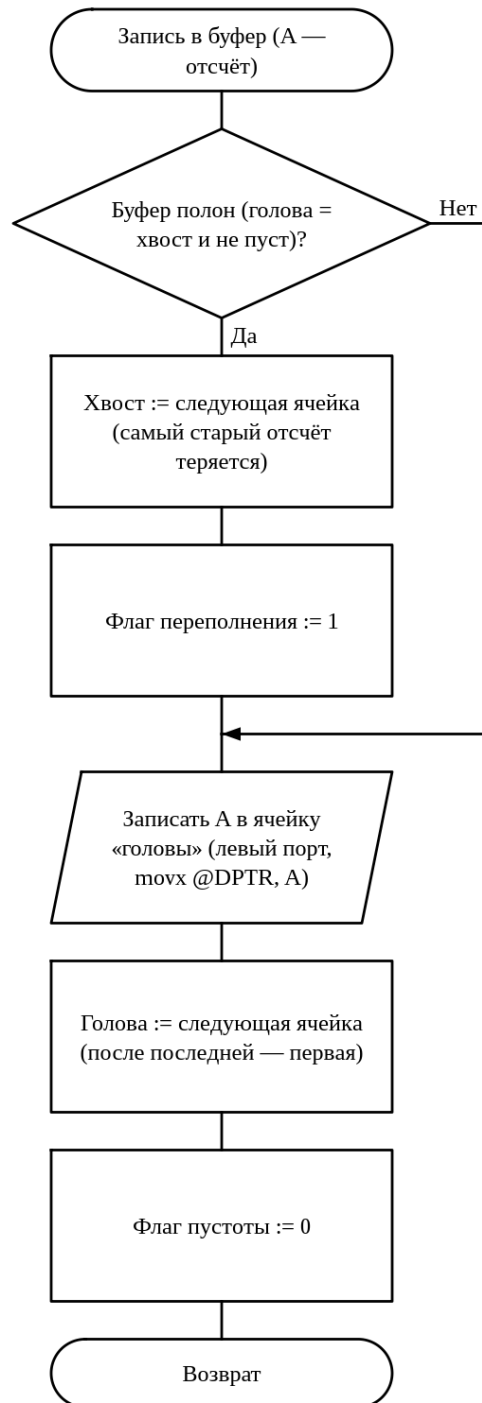


Рис. 3.5. Схема алгоритма: Запись отсчёта в кольцевой буфер

Схема «Запись отсчёта в кольцевой буфер» выполняется так: вход — «Запись в буфер (A — отсчёт)»; проверяется условие «Буфер полон (голова = хвост и не пуст)»: если да — хвост := следующая ячейка (самый старый отсчёт теряется), флаг переполнения := 1; обмен с устройством: записать A в ячейку «головы» (левый порт, `movx @DPTR, A`); голова := следующая ячейка (после последней — первая); флаг пустоты := 0; в конце — возврат.

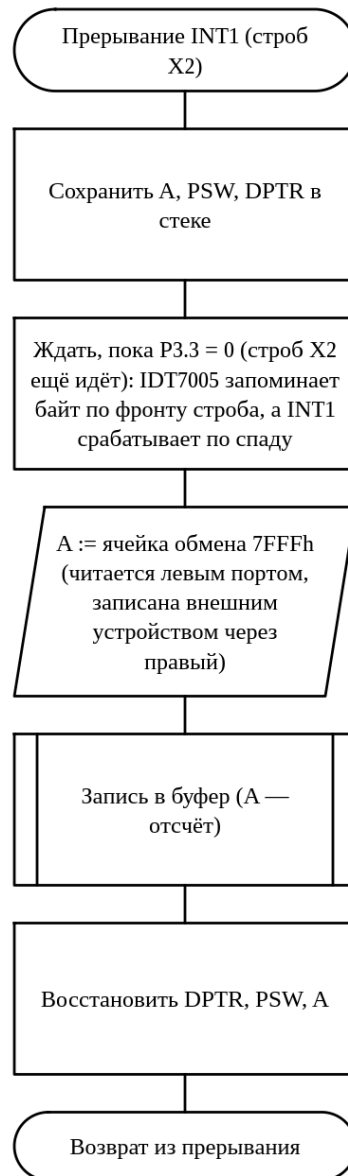


Рис. 3.6. Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

К моменту прерывания отсчёт X2 уже пишется внешним устройством через правый порт IDT7005 в ячейку обмена 7FFFh — МК в этой записи не участвует. Обработчик сохраняет A, PSW и DPTR, потому что вызывается посреди любой другой работы. Затем он крутится на проверке P3.3: INT1 настроен на спад строба, а в ОЗУ байт защёлкивается только по фронту, и если читать сразу, можно получить старое содержимое ячейки. Когда строб закончился, ячейка 7FFFh читается левым портом командой `movx A, @DPTR`, и отсчёт передаётся процедуре записи в кольцо (рисунок 3.5), которая сама сдвигает «голову» и ставит флаги. В конце регистры восстанавливаются в обратном порядке.

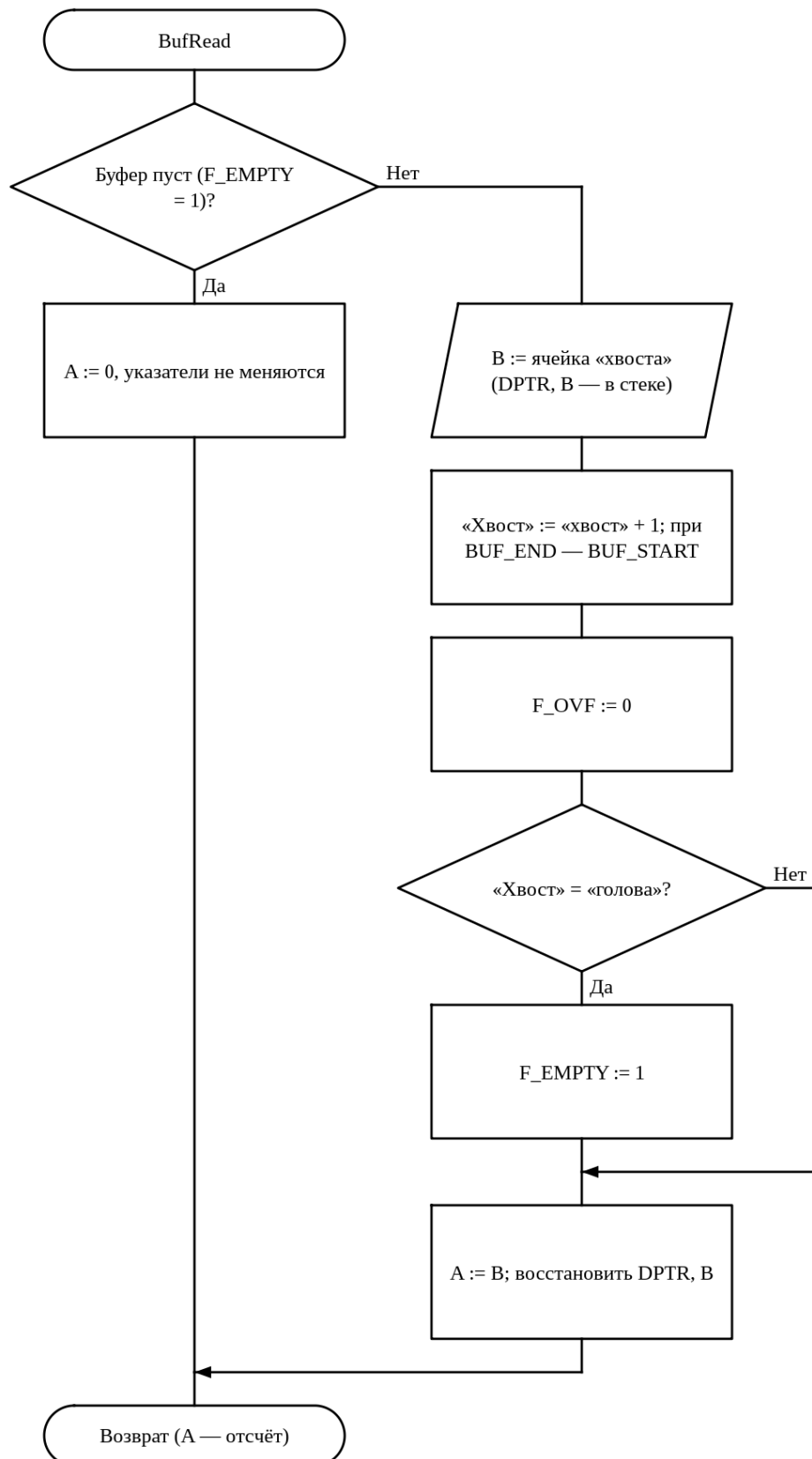


Рис. 3.7. Схема алгоритма: Чтение отсчёта из кольцевого буфера

Сначала проверяется флаг пустоты: если читать нечего, процедура сразу возвращает 0 и не трогает ни указатели, ни флаги. Иначе адрес «хвоста» загружается в DPTR, байт читается через левый порт IDT7005 и откладывается в В, потому что аккумулятор нужен для сравнений. «Хвост» увеличивается на единицу, и если он дошёл до BUF_END = 65F8h (сравниваются оба байта адреса), то возвращается на начало 6000h. После чтения одна ячейка заведомо свободна,

поэтому флаг переполнения сбрасывается, а если «хвост» догнал «голову», ставится флаг пустоты.

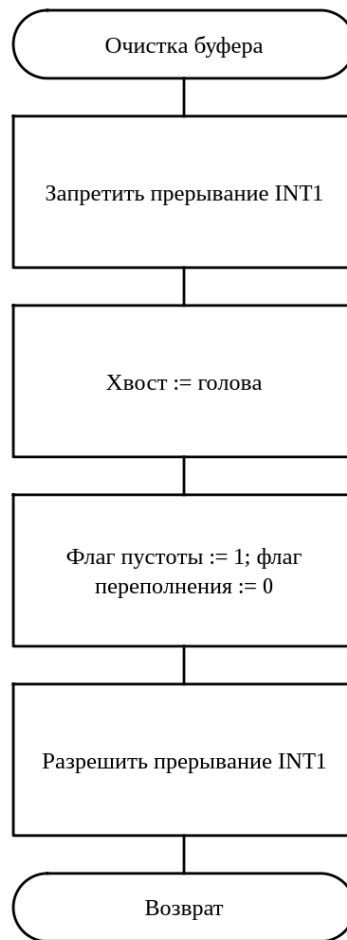


Рис. 3.8. Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

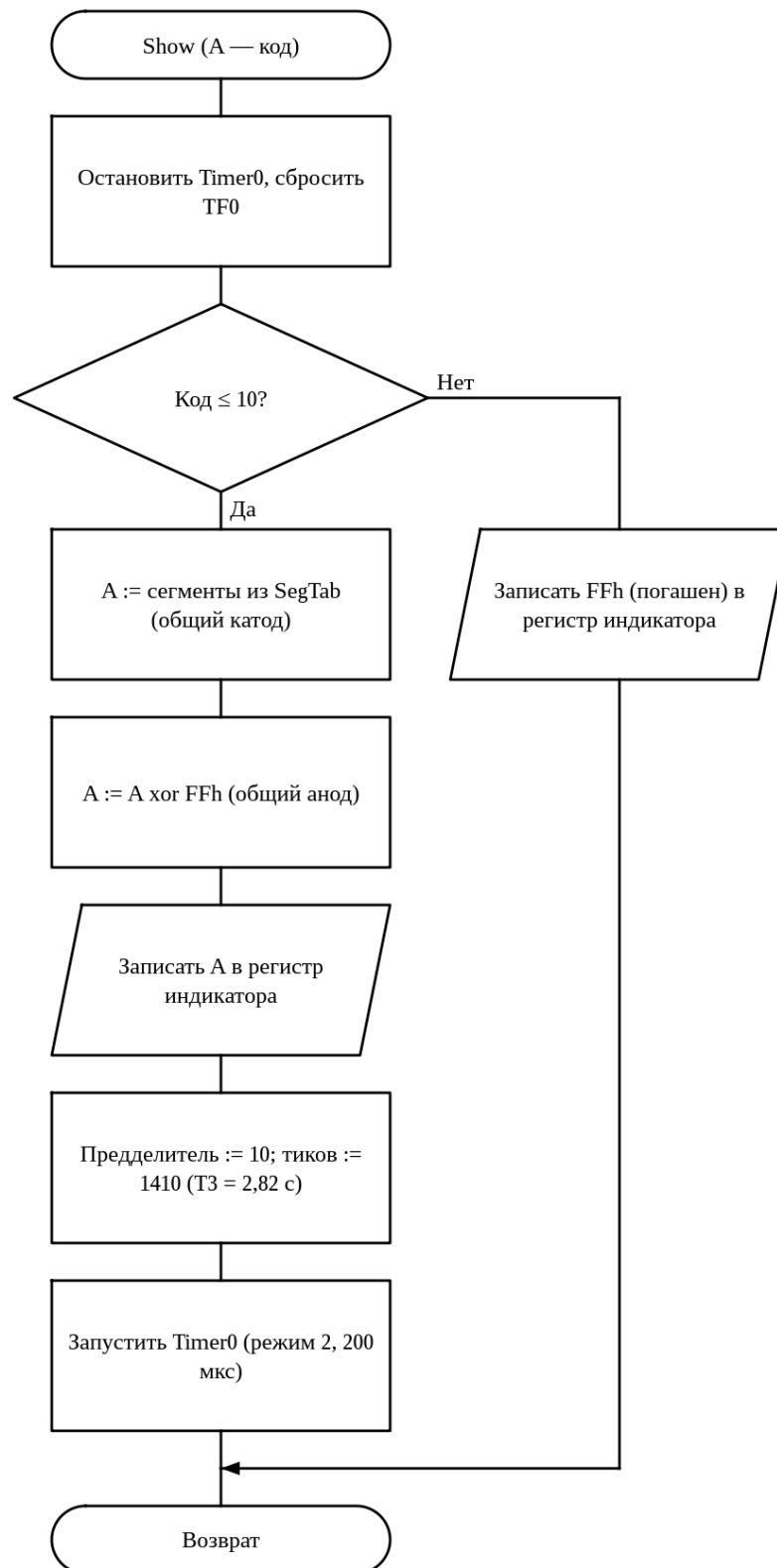


Рис. 3.9. Схема алгоритма: Вывод символа на индикатор

Процедура сначала останавливает Timer0 и сбрасывает TF0: если предыдущий символ ещё показывается, его отсчёт T3 прерывается и начинается заново. Код больше 10 означает «погасить» — в регистр индикатора сразу пишется FFh, и таймер не запускается. Для кодов 0–10 сегменты берутся из таблицы SegTab, составленной для общего катода, и инвертируются командой `xrl A, #IND_OFF`, так как у моего индикатора общий анод и сегмент горит при нуле. После записи символа задаётся счёт тиков T3 (1410 по 2 мс) и запускается Timer0; процедура возвращается сразу, ожидание идёт в прерываниях.

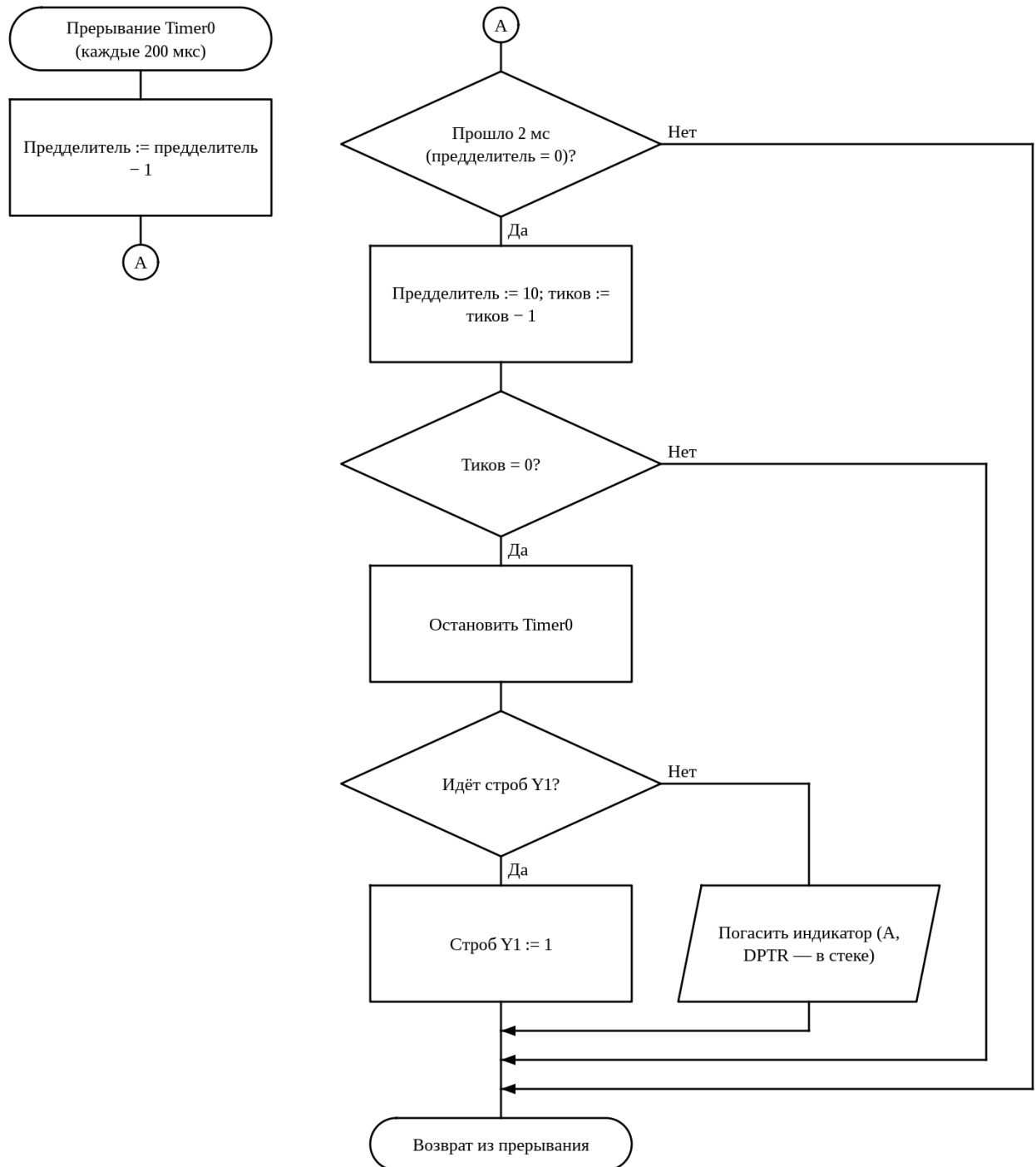


Рис. 3.10. Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Timer0 работает в режиме 2 (8 бит с автоперезагрузкой): переполнение приходит каждые 200 мкс, и перезагружать таймер программно не нужно, поэтому задержка входа в прерывание не копится в длительности. Десять переполнений образуют тик 2 мс, на каждом тике уменьшается двухбайтовый счётчик тиков (младший байт гоняется командой `djnz`, старший — после каждого его круга). Когда тики кончились, таймер останавливается и в зависимости от того, что отсчитывалось, либо снимается строб Y1, либо гасится индикатор. Команды `djnz` не меняют флаги, поэтому PSW сохранять не нужно, а A и DPTR кладутся в стек только в ветке гашения, где они реально используются; в программе 3 используется только ветка T3.

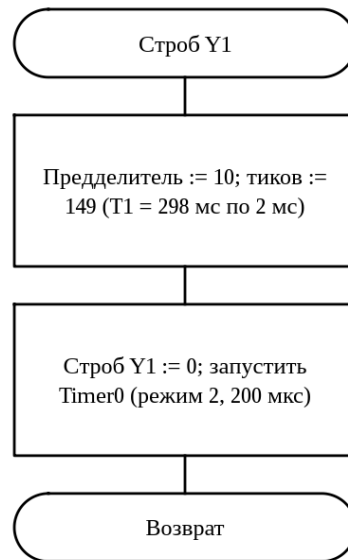


Рис. 3.11. Схема алгоритма: Начало строба Y1

Схема «Начало строба Y1» выполняется так: вход — «Строб Y1»; предделитель := 10; тиков := 149 (T1 = 298 мс по 2 мс); строб Y1 := 0; запустить Timer0 (режим 2, 200 мкс); в конце — возврат.

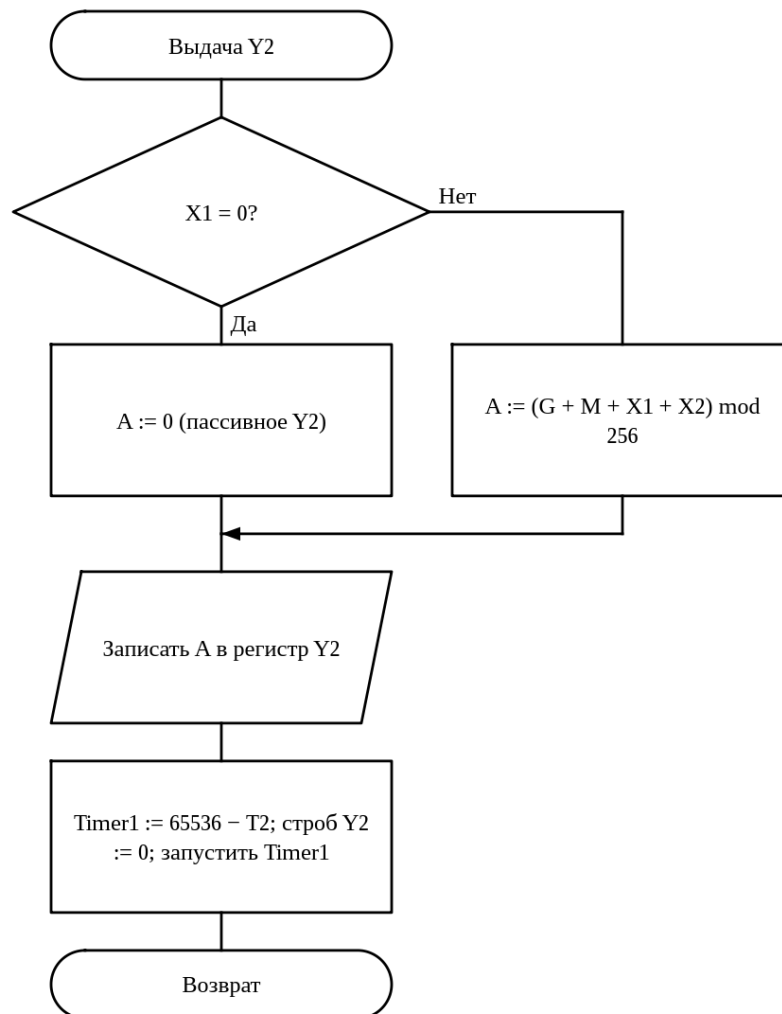


Рис. 3.12. Схема алгоритма: Расчёт и выдача сигнала Y2

Схема «Расчёт и выдача сигнала Y2» выполняется так: вход — «Выдача Y2»; проверяется условие «X1 = 0»: если да — A := 0 (пассивное Y2), иначе — A := (G + M + X1 +

X2) mod 256; обмен с устройством: записать A в регистр Y2; Timer1 := 65536 – T2; строб Y2 := 0; запустить Timer1; в конце — возврат.



Рис. 3.13. Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Схема «Обработчик прерывания Timer1: конец строба Y2» выполняется так: вход — «Прерывание Timer1»; остановить Timer1; строб Y2 := 1; в конце — возврат из прерывания.

3.2. Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос матрицы 4x3 по столбцам, вызывается из обработчика INT0. Каждый столбец опускается по очереди, при этом читаются три строки. Считаем, сколько контактов замкнуто: ровно один — код по табл. 3 ТЗ, иначе 0FFh.

Входные данные: нет.

Выходные данные: A — код клавиши (0-9, T = 10, C = 11) или 0FFh.

Используемые регистры: только A и флаги; R2-R6, B, DPTR сохраняются в стеке.

3.2.2 Подпрограмма BufRead (программа 2)

Назначение: выдать самый старый отсчёт из кольцевого буфера в IDT7005. Если буфер пуст, ничего не меняется и возвращается A = 0. Иначе читается ячейка «хвоста», «хвост» сдвигается (за последней ячейкой — снова первая), флаг переполнения снимается, а если «хвост» догнал «голову» — ставится флаг пустоты.

Входные данные: нет.

Выходные данные: A — отсчёт (0 при пустом буфере); F_EMPTY, F_OVF.

Используемые регистры: A и флаги; DPTR и B сохраняются в стеке.

3.2.3 Подпрограмма Show (программа 3)

Назначение: показать символ на индикаторе и запустить отсчёт ТЗ до гашения. Повторный вызов перезапускает ТЗ заново. Коды больше 10 гасят индикатор сразу.

Входные данные: A — код по табл. 2 ТЗ (0-9 — цифра, 10 — «Е», больше — погасить).

Выходные данные: нет (символ в регистре индикатора, Timer0 считает ТЗ).

Используемые регистры: A и флаги; DPTR сохраняется в стеке.

3.2.4 Обработчик прерывания Timer0 — T0Isr (программа 3)

Назначение: переполнение Timer0 раз в 200 мкс: каждые 10 раз — тик 2 мс, счёт тиков T3. Когда тики кончились — индикатор гасится, таймер останавливается. djnz флаги не меняет, поэтому PSW трогать не нужно; A сохраняется там, где используется.

Входные данные: SubCnt, TicksH:TicksL.

Выходные данные: нет.

Используемые регистры: ничего.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 3.1–3.3.

Таблица 3.1

Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (298 мс) и T_3 (2820 мс), тик 2 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (2068 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 3.2

Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y5	0A000h–0BFFFh
Клавиатура	Y6	0C000h–0DFFFh
Буфер (IDT7005)	Y3	6000h–7FFFh
Регистр Y2	Y4	8000h–9FFFh

Таблица 3.3

Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	40h, 41h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	42h, 43h	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	21h.4 (бит 0Ch)	флаг пустоты буфера
F_OVF	22h.4 (бит 14h)	флаг переполнения буфера
SubCnt	30h	переменная программы 3
TicksL	31h	переменная программы 3
TicksH	32h	переменная программы 3

3.3. Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4. ВХОДНЫЕ ДАННЫЕ

4.1. Клавиатура

Клавиатура — матрица 4×3 (4 столбцов × 3 строк), сканирование по столбцам: в регистр столбцов (адрес 0C000h, разряды D0...D3) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D2, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.1.

Таблица 4.1

Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3	Col4
Row1	«1» — 1	«2» — 2	«3» — 3	«Т» — 10
Row2	«4» — 4	«5» — 5	«6» — 6	«С» — 11
Row3	«7» — 7	«8» — 8	«9» — 9	«0» — 0

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2. Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1528 байт (адреса 6000h–65F7h).

5. ВЫХОДНЫЕ ДАННЫЕ

5.1. Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введенная цифра — до ввода следующей, но не дольше $T_3 = 2,820$ с; при ошибке — код ошибки с горящей точкой др. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 6000h–65F7h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчет затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 298 мс по клавише «Тест». $Y_2 = (8 + 12 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 2068 мкс; до записи регистр выдает предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:TL1 = 65536 – T2 / $T_{мц}$ = 65536 – 2068 = 63468 = 0F7ECh (строб Y_2 , таймер 1)

ТН0:TL0 = 65536 – 2 мс / $T_{мц}$ = 65536 – 2000 = 63536 = 0F830h (тик 2 мс, таймер 0)

$N_{T1} = T1 / 2 \text{ мс} = 149$; $N_{T3} = T3 / 2 \text{ мс} = 1410$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчеты поступают с максимальной интенсивностью):

$t_{\min} = V / f_{X2} = 1528 / 4 = 382,0$ с (6,4 мин).

Максимальная глубина стека для программы 1 — 12 байт (SP от 07h до 13h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2. Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5.1: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dr; индикатор с общим анодом — сегмент горит при 0.

Таблица 5.1

Коды сегментов индикатора (dp g f e d c b a)			
Код	Символ	Двоичный	Шестн.
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«E»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 5.2

Коды ошибок (символ с горящей точкой др, индикация до нажатия любой клавиши, но не дольше ТЗ)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

Таблица описывает реакцию полной системы. В сданных программах из неё реализовано распознавание ошибок клавиатуры: если при опросе замкнуто больше одного контакта или прерывание пришло без нажатия, процедура KbScan возвращает FFh вместо кода клавиши, и вместо цифры на индикатор выводится код ошибки с горящей точкой. Оба случая дают в А одно и то же FFh; различить «1.» и «2.» можно по числу найденных замыканий (больше одного или ни одного), которое процедура считает во время опроса. Переполнение буфера («3.») отмечается флагом F_OVF, который снимает процедура чтения; «4.» выводится, если «Тест» нажат во время строба Y_1 . Индикация ошибки держится до нажатия любой клавиши, но не дольше ТЗ.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 8000h), строб — на выводе P1.6; сигнал Y_1 — на выводе P1.5.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 5.3

Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Лепташкин ВИ-код-1»

```
; Лепташкин В.И., А-08-23, 12, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для работа mpscode вклеит его текст
DSEG AT 30h
; переменные в ОЗУ не нужны: всё держим в регистрах R2-R6 и сохраняем в стеке
CSEG AT 0h
    sjmp START
org 03h ; INT0 — нажатие клавиши: процедура вызывается отсюда
    call KbScan ; %proc%
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h ; стек сразу за банком 0
    setb PIN_CSEN ; разрешить дешифратор CS (без него movx никуда не попадёт)
    mov DPTR, #ADR_KB ; регистр столбцов клавиатуры
    clr A
    movx @DPTR, A ; опустить все столбцы: любое нажатие сразу даст спад на INT0
    setb IT0 ; INT0 срабатывает по спаду
    clr IE0 ; забыть возможный ложный запрос от включения
    setb EX0 ; разрешить прерывание от клавиатуры
    setb EA ; и прерывания вообще
    jmp $ ; %stop%

; KbScan — опрос матрицы 4x3 по столбцам, вызывается из обработчика INT0.
; Каждый столбец опускается по очереди, при этом читаются три строки.
; Считаем, сколько контактов замкнуто: ровно один — код по табл. 3 ТЗ, иначе 0FFh.
; Вход: нет. Выход: A — код клавиши (0-9, T = 10, C = 11) или 0FFh.
; Портит: только A и флаги; R2-R6, B, DPTR сохраняются в стеке.
KbScan:
    push PSW
    push DPL
    push DPH
    push AR2
    push AR3
    push AR4
    push AR5
    push AR6
    mov DPTR, #ADR_KB ; одна ячейка: запись — столбцы, чтение — строки
    mov R3, #0 ; номер клавиши (строка * 4 + столбец) последнего замыкания
    mov R4, #0 ; сколько замыканий нашли за весь опрос
    mov R5, #0FEh ; маска: ноль в бите опрашиваемого столбца
    mov R2, #0 ; номер текущего столбца, 0..3

KbCol:
    mov A, R5
    movx @DPTR, A ; опустить только этот столбец
    movx A, @DPTR ; снять состояние строк
    anl A, #(1 SHL KB_ROWS) - 1 ; старшие биты шины не подключены — отбросить
    mov R6, AR2 ; номер клавиши в первой строке равен номеру столбца
    mov B, #KB_ROWS ; сколько строк перебрать

KbRow:
    rrc A ; бит очередной строки уходит в C
    jc KbRowNext ; единица — контакт разомкнут
    inc R4 ; нашли ещё одно замыкание
    mov R3, AR6 ; запомнить, какая это клавиша

KbRowNext:
    xch A, R6 ; номер клавиши для строки ниже
    add A, #KB_COLS ; больше на число столбцов
    xch A, R6
    djnz B, KbRow
    mov A, R5 ; ноль в маске переносим на следующий столбец
    rl A
    mov R5, A
    inc R2
    cjne R2, #KB_COLS, KbCol
    clr A
```

```

movx @DPTR, A          ; опрос окончен — снова опустить все столбцы для INT0
cjne R4, #1, KbNone    ; ни одной или две и больше — ошибка
mov A, R3
mov DPTR, #KbCodes
movc A, @A+DPTR        ; номер клавиши -> код по табл. 3 ТЗ
sjmp KbOut

KbNone:
mov A, #0FFh           ; помеха или несколько клавиш сразу

KbOut:
clr IE0                ; пока перебирали столбцы, INT0 дёргался — сбросить лишний запрос
pop AR6
pop AR5
pop AR4
pop AR3
pop AR2
pop DPH
pop DPL
pop PSW
ret

; Коды клавиш по рис. 4 ТЗ (4 столбца x 3 строки), строка за строкой
KbCodes:
DB 1, 2, 3, 10         ; Row1: 1 2 3 Т
DB 4, 5, 6, 11         ; Row2: 4 5 6 С
DB 7, 8, 9, 0          ; Row3: 7 8 9 0

END

```

Программа 2 — файл «Лапташкин ВИ-код-2»

```

; Лапташкин В.И., А-08-23, 12, v1
$INCLUDE (vars.inc)      ; адреса, биты, константы варианта; в файл для работа mpcode вклеит его текст
DSEG AT 30h
; своих переменных нет: указатели и флаги буфера заданы в vars.inc
CSEG AT 0h
sjmp START
org 03h ; "заглушка" для INT0
nop
reti
org 0bh ; "заглушка" для Tmr0_ovf
nop
reti
org 13h ; "заглушка" для INT1
nop
reti
org 1bh ; "заглушка" для Tmr1_ovf
nop
reti
org 23h ; "заглушка" для UART
nop
reti
org 2bh
START:
; Инициализация МК *****
mov SP, #07h           ; стек сразу за банком 0
setb PIN_CSEN          ; разрешить дешифратор CS, иначе буфер не выбирается
mov HEAD_L, #LOW(BUF_START) ; «голова» — на первую ячейку буфера
mov HEAD_H, #HIGH(BUF_START)
mov TAIL_L, #LOW(BUF_START) ; «хвост» — туда же: буфер пуст
mov TAIL_H, #HIGH(BUF_START)
setb F_EMPTY          ; отсчётов нет
clr F_OVF              ; переполнения не было
call BufRead ; %proc%
jmp $ ; %stop%

; BufRead — выдать самый старый отсчёт из кольцевого буфера в IDT7005.
; Если буфер пуст, ничего не меняется и возвращается A = 0.
; Иначе читается ячейка «хвоста», «хвост» сдвигается (за последней ячейкой —
; снова первая), флаг переполнения снимается, а если «хвост» догнал «голову» —
; ставится флаг пустоты.
; Вход: нет. Выход: A — отсчёт (0 при пустом буфере); F_EMPTY, F_OVF.
; Портит: A и флаги; DPTR и B сохраняются в стеке.
BufRead:
jnb F_EMPTY, BrEmpty   ; читать нечего
push DPL
push DPH
push B
mov DPL, TAIL_L         ; адрес самого старого отсчёта
mov DPH, TAIL_H
movx A, @DPTR           ; взять отсчёт из буфера
mov B, A               ; отложить, A нужен для сравнений
inc DPTR               ; следующая ячейка
mov A, DPL              ; вышли за конец буфера? сверяем оба байта
cjne A, #LOW(BUF_END), BrSave
mov A, DPH
cjne A, #HIGH(BUF_END), BrSave

```

```

        mov DPTR, #BUF_START      ; заворот на начало кольца
BrSave:
        mov TAIL_L, DPL           ; запомнить новый «хвост»
        mov TAIL_H, DPH
        clr F_OVF                 ; одно место освободилось — буфер не полон
        mov A, TAIL_L             ; «хвост» догнал «голову» — всё прочитано
        cjne A, HEAD_L, BrDone
        mov A, TAIL_H
        cjne A, HEAD_H, BrDone
        setb F_EMPTY

BrDone:
        mov A, B                  ; вернуть отсчёт
        pop B
        pop DPH
        pop DPL
        ret

BrEmpty:
        clr A                     ; пустой буфер: условный ноль, указатели не трогаем
        ret

END

```

Программа 3 — файл «Лапташкин ВИ-код-3»

```

; Лапташкин В.И., А-08-23, 12, v1
$INCLUDE (vars.inc)              ; адреса, биты, константы варианта; в файл для робота mpcode вклеит его текст
; Timer0 в режиме 2 (8 бит с автоперезагрузкой): переполнение каждые 200 мкс без программной перезагрузки,
; 10 переполнений = один тик 2 мс из vars.inc, T3 = T3_TICKS таких тиков
SUB_US EQU 200                   ; период переполнения Timer0, мкс (1 машинный цикл = 1 мкс)
SUB_N EQU 10                     ; переполнений в одном тике 2 мс
DSEG AT 30h
SubCnt: DS 1                     ; сколько переполнений по 200 мкс осталось до конца тика
TicksL: DS 1                     ; счётчик тиков T3 для djnz: младший байт
TicksH: DS 1                     ; старший байт (сколько раз пройти младший до нуля)
CSEG AT 0h
        sjmp START
org 03h ; "заглушка" для INT0
        nop
        reti
org 0bh ; Timer0 — тик для отсчёта T3
        ljmp T0Isr
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh ; "заглушка" для Tmr1_ovf
        nop
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК *****
        mov SP, #07h             ; стек сразу за банком 0
        setb PIN_CSEN            ; разрешить дешифратор CS
        mov DPTR, #ADR_IND
        mov A, #IND_OFF
        movx @DPTR, A           ; при включении индикатор тёмный
        mov TMOD, #02h          ; Timer0 — режим 2, перезагрузка из TMO аппаратно
        mov TH0, #256 - SUB_US   ; значение перезагрузки: 200 счётов до переполнения
        clr TR0                  ; пока символа нет — таймер стоит
        clr TF0
        setb ET0                 ; прерывание от Timer0
        setb EA
        mov A, #5                ; пример: показать «5»
        call Show ; %proc%
        jmp $ ; %stop%

; Show — показать символ на индикаторе и запустить отсчёт T3 до гашения.
; Повторный вызов перезапускает T3 заново. Коды больше 10 гасят индикатор сразу.
; Вход: A — код по табл. 2 T3 (0-9 — цифра, 10 — «Е», больше — погасить).
; Выход: нет (символ в регистре индикатора, Timer0 считает T3).
; Портит: A и флаги; DPTR сохраняется в стеке.
Show:
        push DPL
        push DPH
        clr TR0                  ; прежний отсчёт T3 больше не нужен
        clr TF0                  ; и его недосчитанный запрос тоже
        cjne A, #11, ShCmp       ; C = 1, если код меньше 11
ShCmp:
        jnc ShOff                ; кода такого нет — просто погасить
        mov DPTR, #SegTab
        movc A, @A+DPTR          ; сегменты для общего катода (1 — горит)
        xrl A, #IND_OFF          ; общий анод: сегмент зажигает 0, поэтому инвертируем
        mov DPTR, #ADR_IND
        movx @DPTR, A           ; символ в регистр индикатора

```

```

mov SubCnt, #SUB_N      ; новый тик начинается с нуля
mov TicksL, #LOW(T3_TICKS) ; счёт под djnz: сначала младший байт до нуля,
mov TicksH, #HIGH(T3_TICKS - 1) + 1 ; потом по 256 на каждую единицу старшего
mov TL0, #256 - SUB_US   ; первое переполнение через 200 мкс
setb TR0                  ; пошёл отсчёт
sjmp ShRet

ShOff:
mov DPTR, #ADR_IND
mov A, #IND_OFF
movx @DPTR, A             ; погасить все сегменты

ShRet:
pop DPH
pop DPL
ret

; T0Isr – переполнение Timer0 раз в 200 мкс: каждые 10 раз – тик 2 мс, счёт тиков T3.
; Когда тики кончились – индикатор гасится, таймер останавливается.
; djnz флаги не меняет, поэтому PSW трогать не нужно; A сохраняется там, где используется.
; Вход: SubCnt, TicksH:TicksL. Выход: нет. Портит: ничего.
T0Isr:
djmp SubCnt, T0Exit      ; тик 2 мс ещё не набрался
mov SubCnt, #SUB_N      ; следующий тик
djmp TicksL, T0Exit      ; минус один тик
djmp TicksH, T0Exit      ; младший байт прошёл круг – минус 256 тиков
clr TR0                  ; T3 истекло – больше не считать
push ACC
push DPL
push DPH
mov DPTR, #ADR_IND
mov A, #IND_OFF
movx @DPTR, A           ; гасим индикатор
pop DPH
pop DPL
pop ACC

T0Exit:
reti

; Сегменты цифр 0-9 и «Е» для общего катода: D0..D7 = a, b, c, d, e, f, g, dp
SegTab:
DB 3Fh, 06h, 5Bh, 4Fh, 66h ; 0 1 2 3 4
DB 6Dh, 7Dh, 07h, 7Fh, 6Fh ; 5 6 7 8 9
DB 79h                      ; E

END

```